

TARTU ÜLIKOOL
Arvutiteaduse instituut
Informaatika õppekava

Uku Renek Kronbergs
**Generatiivse tehisintellekti mõju
tarkvaraarenduse produktiivsusele algajate
programmeerijate seas**
Bakalaureusetöö (9 EAP)

Juhendaja:
Alo Peets, MSc

Tartu 2026

Generatiivse tehisintellekti mõju tarkvaraarenduse produktiivsusele algajate programmeerijate seas

Lühikokkuvõte:

Käesolev bakalaureusetöö uurib suuremahuliste keelemudelite (nt ChatGPT, GitHub Copilot) ehk generatiivse tehisintellekti (GenAI) mõju tarkvaraarenduse produktiivsusele algajate programmeerijate seas. Seoses tehisintellekti tööriistade laialdase levikuga on tekkinud vajadus mõista, kuidas need vahendid muudavad koodi kirjutamise ja silumise protsessi, eriti nende kasutajate hulgas, kellel puudub pikaajaline arenduskogemus. Töö käigus viidi veebipõhises keskkonnas läbi kvantitatiivne uuring ja koodianalüüs 30 informaatikatiudengi osalusel, keda jaotati lokaalse tehisintellekti tööriista kasutajate ja kontrollgrupi vahel. Tulemused näitavad, et GenAI tööriistade kasutamine kiirendab oluliselt esialgse koodibaasi loomist, kuid koodi turvalisuse tagamine ning loogikavigade leidmine võtab algajatel sageli varasemast rohkem aega, osutades vajadusele arendada süsteemsemat arusaama GenAI väljundi kriitiliseks hindamiseks.

Võtmesõnad: Generatiivne tehisintellekt, informaatika haridus, programmeerimine, tarkvaraarenduse produktiivsus, suured keelemudelid

CERCS: P175 Informaatika, süsteemiteooria

The Impact of Generative AI on Software Development Productivity Among Novice Programmers

Abstract:

This Bachelor's thesis investigates the impact of large language models (e.g., ChatGPT, GitHub Copilot), collectively referred to as generative AI (GenAI), on software development productivity among novice programmers. With the widespread adoption of AI tools, there is a growing need to understand how these technologies alter the process of writing and debugging code, particularly for users lacking extensive development experience. A quantitative study and code analysis were conducted in a web-based environment involving 30 computer science students, who were divided into an experimental group using local AI tools and a control group. The results indicate that the use of GenAI tools significantly accelerates the creation of the initial codebase; however, ensuring code security and identifying logic errors often takes novices more time than traditional methods, highlighting the need to develop a more systematic understanding for critically evaluating GenAI output.

Keywords: Generative artificial intelligence, computer science education, programming, software development productivity, large language models

CERCS: P175 Informatics, systems theory

Sisukord

1. Sissejuhatus	5
1.1 Probleemi püstitus ja töö eesmärk	6
1.2 Töö originaalne panus.....	7
1.3 Mõisted ja lühendid.....	7
1.4 Töö struktuur	8
2. Taust ja kirjandusülevaade	9
2.1 Koodi genereerivate tehisintellektide arengulugu	9
2.2 Suurte keelemudelite sisemine arhitektuur	10
2.3 Programmeerimise õppimine: Kognitiivne jaotuvus.....	11
2.4 Kriitikavaba vastuvõtt ja over-reliance	11
2.5 Varasemad eksperimentaalsed uurinud AI ja produktiivsuse vahel	12
3. Metoodika ja eksperimendi disain	14
3.1 Uuringu disain ja peamised hüpoteesid.....	14
3.2 Valim ja osalejate demograafia	14
3.3 Testikeskkonna tehniline arhitektuur.....	15
3.4 Testülesannete spetsifikatsioon	16
3.5 Katse läbiviimise protseduur	16
3.6 Andmekaitse ja eetika.....	17
4. Tulemused ja arutelu	18
4.1 Kvantitatiivne analüüs: Ülesannete soorituskiirus (Ajakulu)	18
4.2 Koodi kvaliteet, äärejuhud ja turvalisus (Edukus).....	19
4.3 Kvalitatiivne tagasiside: Tudengite isiklikud peegeldushinnangud	21
5. Kokkuvõte.....	22
5.1 Uuringuküsimuste järelused	22
5.2 Uuringu piirangud ja edasised perspektiivid	23
Viited.....	24
Lisad	25
Litsents	39

1. Sissejuhatus

Tarkvaraarenduse valdkond on viimaste aastate jooksul teinud läbi märkimisväärse arengutõuke tänu erinevate tehisintellekti vormide laialdasele levikule ja kättesaadavusele. Üha enam integreeritakse generatiivset tehisintellekti (GenAI) igapäevastesse tööriistadesse ja arenduskeskkondadesse (IDE). Tööriistad nagu OpenAI poolt arendatud *ChatGPT*, *GitHub Copilot* ja erinevad lokaalsed suured keelemudelid (näiteks *LLaMA* perekond) on muutunud oluliseks abivahendiks paljudele tarkvaraarendajatele. Selliste lahenduste esmane eesmärk tööturul on tarkvara kättetoimetamise elutsükli (SDLC) kiirendamine ning kogenud arendajate senise töövoos rutiinsete osade optimeerimine – olgu selleks siis standardse süntaksi või konfiguratsioonifailide loomine, levinud veateadete lahendamine, testide kirjutamine või isegi suuremahuliste arhitektuuriliste probleemide skemaatiline lahendamine erinevates programmeerimiskeeltes [1].

Samal ajal kerkib infotehnoloogia õpetamise (didaktika) ja hariduse vaatenurgast esile väga kriitiline küsimus: mil määral on need võimsad tööriistad abiks *algajatele* programmeerijatele, kes ei ole veel süsteemselt omandanud ei programmeerimise baaskontseptsioone (nagu muutujad, objektorienteeritud disain, andmestruktuurid) ega ka tarkvara turvalise arhitektuuri aluseid? Ühelt poolt on tänasel IT erialade esmakursuslasel kohene ligipääs tehisintellektile, mis suudab tänu loomuliku keele töötlusele lahendada igasuguse algtaseme programmeerimisülesande vaid mõne sekundiga [2]. Õpilase vaatenurgast kaob sageli ära frustratsioon, mis muidu tekib kompileerimisvigadega maadlemisel ja dokumentatsiooni puudulikkuse tõttu, sest tehisintellekt ei genereeri mitte ainult lahenduse, vaid pakub koheselt välja ka veaparanduse ideed ja selgitused. See asjaolu toetab nii iseseisva õppimise suutlikkust kui ka lahenduskäikude avastamist.

Kuid selle potentsiaali kõrval on tekkinud ja kaardistatud ka tõsised ohud. Paljud teadlased viitavad nähtusele, kus algajad hakkavad nii tugevalt toetuma tehisintellektile (ingl. *over-reliance*), et nad kaotavad võime iseseisvalt ja kriitiliselt probleeme lahendada. Õppija kognitiivne fookus kandub algoritmilise mõtlemise arendamiselt üle viipetehnika (ingl. *prompt engineering*) haldamisele, kus koodi loomine asendub peamiselt mudeli tagastatud teksti kopeerimisega [3]. Seeläbi võivad jääda pinnapealseks isegi kõige lihtsamate andmestruktuuride operatsioonid, mis hiljem peadib fundamentaalse suutmatusega analüüsida ja siluda keerukamaid, skaleeruvaid süsteeme, mida mudel ise korraga hoomata ei pruugi. Lisaks, tehisintellekt "hallutsineerib" – genereerib lahendusi, mis on loogiliselt vigased või sisaldavad sügavaid turvavigu. Kui õpilasel

puudub varasem kogemus koodi kriitiliseks auditeerimiseks, võivad need vead jääda neile nähtamatuks ja kujuneda õpiprotsessis juurdunud väärteks mustriteks.

1.1 Probleemi püstitus ja töö eesmärk

Seistes silmitsi selle tehnoloogilise murranguga, peavad ülikoolid ja teised haridusasutused kujundama oma õppematerjalid ümber, et leida tasakaal, mis võimaldaks üliõpilastel valmistuda GenAI-ajastu tööturaks, ilma et nad ohverdaksid tehniliste oskuste baasvundamendi. Eestis, kus infotehnoloogia on majanduse oluline vedur ja IT-haridus laialdaselt levinud, puudub aga hetkel veel piisavalt laiapõhjaline empiiriline andmestik, mis näitaks detailselt esmakursuslaste käitumist ja produktiivsust reaalsetes tingimustes tehisintellektiga interakteerumisel.

Käesoleva bakalaureusetöö eesmärk on just selle andmelünga osaline täitmine kohalikus kontekstis. Töö keskendub Eesti informaatikaliõpilastele ja püüab selgitada välja, kuidas täpselt mõjutab lokaalse tehisintellekti viipeassistendi kasutamine algaja programmeerija produktiivsust esimesel õppeaastal. Tulemuste analüüsimisel keskendutakse eelkõige kirjutatud koodi funktsionaalsusele ja õpilaste ajalistele ressurssidele probleemide dekomponeerimisel. Töös püstitati järgnevad kaks peamist uurimisküsimust (UK):

1. **UK1:** Kas tehisintellekti abiga (katsegrupp) lahendatud baastaseme programmeerimisülesannete funktsionaalse lahenduse ni jõudmise kiirus on statistiliselt oluliselt kiirem võrreldes traditsiooniliselt, vaid dokumentatsiooni toel (kontrollgrupp) lahendatud ülesannetega?
2. **UK2:** Millises proportsioonis esineb tehisintellekti abiga loodud koodides spetsiifilisi äärejuhtumite (ingl. *edge-cases*) eiramise loogikavigu ja potentsiaalseid turvariske võrreldes manuaalselt koostatud lahendustega, ning mil määral on tudengid suutelised iseseisvalt neid vigu tuvastama?

Töö eesmärgi täitmiseks konstrueeris autor 2026. aasta kevadsemestril spetsiaalse monitooritava veebipõhise programmeerimiskeskonna, mis võimaldas viia läbi kvantitatiivse eksperimendi Tartu Ülikooli uute tudengite seas. Erinevalt tavapäraest uurimustest, mis põhinevad vaid ankeetküsitlustel, jälgis loodud testikeskkond arendajate tegevust sekundi ja klahvivajutuse täpsusega (sh kopeerimised, käivitamise tulemused, esitatud viibad mudelile), pakkudes rikkalikku andmestikku mõistmaks arendaja ja AI vahelist interaktsiooniprotsessi detailsemalt.

1.2 Töö originaalne panus

Käesoleva töö teaduslik ja rakenduslik panus seisneb mitmes aspektis:

- **Empiiriline tugi:** Töös esitatakse uudne, Eesti ülikoolide kontekstis kogutud andmestik, mis võrdleb kvantitatiivselt kontroll- ja katsegruppide edasijõudmist tehisintellekti tugilahendustega.
- **Tehniline teostus:** Katsetusteks loodud spetsiaalne kliendi-server arhitektuuriga testilahendus on analüüsivõimekas tööriist, mille saaks potentsiaalselt edaspidi integreerida ülikooli õpikeskkondadesse ja varase hoiatamise süsteemidesse.
- **Pedagoogiline sisend:** Töö tulemused annavad sisendi sissejuhatavate arvutiteaduse kursuste metoodika revideerimiseks (näiteks automaatsete ja eksamitingimuste karmistamine).

1.3 Mõisted ja lühendid

Alljärgnevalt on lahti seletatud töös kesksel kohal olevad spetsiifilised mõisted ja akronüümid, mida kasutatakse järgnevates peatükkides korduvalt.

AI (Tehisintellekt) - *Artificial Intelligence*. Masinate (peamiselt arvutisüsteemide) simuleeritud kognitiivne funktsioon, mis suudab lahendada probleeme ja õppida.

GenAI - Generatiivne tehisintellekt (*Generative Artificial Intelligence*). Tehisintellekti alamharu, mis on optimeeritud üle andmestike uue sisu (teksti, pildi, koodi) loomiseks.

LLM (Suur keelemudel) - *Large Language Model*. Miljarditel parameetritel töötav südamikuga narvivõrgu tuum, mis on treenitud hiiglaslikul hulgal tekstiinfot prognoosimaks sõnade järjestikusi tõenäosusi. Näited: GPT-4, Llama 3, Claude.

IDE - *Integrated Development Environment* ehk integreeritud arenduskeskkond (nt Visual Studio Code, PyCharm), mis hõlmab sageli tehisintellekti laiendusi (nagu GitHub Copilot ja CodeWhisperer).

Prompt - Viip. Kasutaja sisestatud tekstiline käsklus või küsimus tehisintellekti mudelile, et tekitada soovitud väljund ja kontekst.

Over-reliance - Ülemäärane toetumine tehisintellektile. Psühholoogiline ja käitumuslik protsess, kus isik loobub kriitilisest kontrollist tööriista väljundi hindamisel ega kohanda oma

varasemat enesedistsipliini, eeldades ekslikult, et pakutud lahendus on alati autoriteetselt korrektne.

Edge case - Äärejuht. Sisendandmete äärmuslik olukord või algoritmiline piirsis, mis ei pruugi avalduda sagedaste kasutusstsenaariumide juures, kuid võib tingida ootamatuid süsteemirikked, kui nendega pole algse disaini faasis arvestatud (nt jagamine nulliga, listi indeksi ületamine, või tühjad sisendid massiivi töötlemisel).

1.4 Töö struktuur

Käesolev bakalaureusetöö on jaotatud viieks suuremaks peatükiks, et tagada teema loogiline ülesehituse järgnevus: Peatükk 1 (Sissejuhatus) annab ülevaate uuritava probleemi aktuaalsusest, defineerib peamised suunad, sõnastab keske uurimisküsimuse ja olulisemad lühendid, mis panevad vundamendi järgnevale käsitlemisele. Peatükk 2 (Taust ja kirjandusülevaade) siseneb teemasse detailselt läbi ajaloooprismade. See analüüsib tehisintellekti arengulugu alates lihtsatest markovi ahelatest kuni Transformer-arhitektuurini. Ühtlasi vaadeldakse eelnevaid rahvusvahelisi uuringuid ja pedagoogilisi käsitlusi, andes lugejale selge arusaama Generatiivsete keelemudelite spetsiifikast. Peatükk 3 (Metoodika ja eksperimendi disain) kirjeldab tehnilist arhitektuuri ja eksperimentaalset protsessi. Vaatleme testgruppi, uuringu keskkonna tehnilist ülesehitust (alglaadimise skripte, andmebaaside struktuure), testülesannete sisu ja andmekogumise kriteeriume läbi privaatsusfiltrite. Peatükk 4 (Tulemused ja arutelu) keskendub logidest genereeritud metaandmete sünteetilisele hindamisele. Leitakse statistilisi seoseid ajakulu, edukate lahenduste ja avastamata koodivigade (sh turvavigade) vahel erinevate uuringurühmade seas. Esindatud on reaalsete olukordade koodilõigud. Tulemusi seostatakse ka eelnevate aastate teaduskirjandusega, et hinnata teooria paikapidavust Eestis läbiviidud eksperimendi põhjal. Lõpuks Peatükk 5 (Kokkuvõte) sünteesib saadud järeldused ning vastab esialgsetele uurimisprobleemidele. Sektsioon heidab pilgu ka tehtud viitekorrektureidele ja uuringu piirangutele koos aknameetritega tulevaste uuringute radadeks. Töö lisades (Lisad) on täies ulatuses ära toodud testikeskkonna esiosa lähtekoodid ja analüüsiks loonud algoritmid, tagamaks andmete auditeeritavuse ning iseseisva reprodutseeritavuse teistele teadlastele.

2. Taust ja kirjandusülevaade

Generatiivse tehisintellekti tehnoloogia, kuigi viimase kahe aasta jooksul kogenud eksponentsiaalset kasvu kasutatavuses ja avalikus tuntuses, ei ole tegelikult üleöö tekkinud fenomen. Selle juured peituvad aastakümneid kestnud statistilises masinõppes ja keele töötluses. Antud peatükk annab süvitsi mineva ülevaate sellest, milline on olnud tehisintellekti evolutsioon programmikoodi genereerimisel, kuidas spetsiifilised algoritmid fundamentaalsel tasemel töötavad ning kuidas suhestub selline mehhanism inimpsühholoogia, täpsemalt õppimise ja kognitiivse ülekoormuse teooriaga. Peatüki lõpp koondab varasemad läbiviidud suurte uuringute andmed sarnaste probleemide lahendamisel.

2.1 Koodi genereerivate tehisintellektide arengulugu

Programmide automaatse genereerimise idee on informaatika valdkonnas sama vana kui programmeerimiskeeled ise. Juba 1960. aastatel unistati süsteemidest, kuhu inimene sisestab loomulikus keeles nõuded ja arvuti kompileerib nendest süsteemi (tuntud kui *Automatic Programming*). Kuid esimesed kümnendid ebaõnnestusid paljuskki loomuliku keele ambivalentsuse ja semantilise mitmetimõistetavuse tõttu, pakkudes töötavaid tulemusi vaid äärmiselt rangetes domeenides loodud reeglipõhiste puudega (ingl. *rule-based expert systems*).

Järjekindel edasimineku keeletöötluses (*Natural Language Processing* - NLP) tabas valdkonda 2010. aastatel läbi rekurrentsete närvivõrkude (RNN) ning pika/lühiajalise mälu (LSTM - *Long Short-Term Memory*) struktuuride. Need tehnoloogiad võimaldasid mudelitel hoida enda sees konteksti, ent piirdusid enamasti lühikeste, mõnelauseliste tekstimassiivide sidusa analüüsiga [4]. Tõeline paradigma muutus tarkvaraarenduse automatiseerimises saabus aga 2017. aastal, mil Google'i teadlased avalikustasid artikli "Attention Is All You Need", tuues massidesse **Transformer-arhitektuuri**.

Transformer võimaldas analüüsida tekstimassiive hiiglaslikul tasandil asünkroonselt tunduvalt laiemas kontekstiaknaga tänu *Self-Attention* (enese-tähelepanu) mehhanismile. Tähelepanumehhanism arvutab igale sisendsõnale kaalu kõikide teiste lauses (või programmikoodis) olevate sõnade (või spetsiifiliste funktsiooninimede, muutujate) suhtes, määrates millisele varasemale kontekstile mudel ennustamisel peab kõige rohkem keskenduma. Pärast sellise vundamendi teket hakkas OpenAI arendama oma GPT (*Generative Pre-trained Transformer*) mudelite ridu. Aastal 2021 tuli GitHub ja OpenAI ühiselt välja spetsiaalselt programmeerimisele treenitud Codex mudeliga, mis pandi aluseks maailma esimesele laiatarbe

tehisintellekti arendusassistendile **GitHub Copilot** [5]. Samasse ajastusse jookseb ka ChatGPT, Llama ja erinevate spetsiifiliste koodigeneraatorite (näit. *StarCoder*) esiletõus. Need mudelid suudavad lisaks ridade lõpetamisele (*code completion*) teostada koodi analüüsi, genereerida algoritmide selgitusi ja pakkuda teste kümnetes miljonites avatud lähtekoodiga (*open-source*) materjalide baasil saadud teadmusest.

2.2 Suurte keelemudelite sisemine arhitektuur

Selleks, et mõista süvitsi, miks AI generaatorid teevad spetsiifilisi ekslikke lahendeid, mida algajad tuvastada ei oska, peab detailsemalt tundma nende sisemisi operatsioone. Suured keelemudelid põhinevad statistilisel tõenäosusarvutusel. Kui kasutaja sisestab arenduskeskkonda oma päringu või palve lahendada mõni Pythoni def-funktsioon, teisendatakse algne sissetulev sõnum ja senine keskkonna aktiivse faili kood sadadeks tuhandeteks **tokeneks** (mis pole ei sõnad ega tähekombinatsioonid otseselt, vaid arvulised vektorruumi kordinaadid vastavas embeddingute ruumis).

Näiteks koodirida `'for i in range(10):'` võib olla mudeli jaoks jaotatud mitmeks sõltumatuks tokeniks ja kaardistatud tema globaalsesse "koodisõnastikku". Self-attention mehhanism töötleb kõiki pika sisendakna (tänapäeval tihti üle 100,000 tokenit) asukohti samaaegselt ning annab süntaksile seletava tähendusliku lingi. Seejärel algab autonoomne regressiivne ennustamine. Mudel otsustab, milline üksik token peaks olema viimasele punktile (*promptile*) statistiliselt kõige ootuspärasem jätk, lisanud genereeritud tokeni sisendina uuesti konteksti ning kordab arvamist tuhandeid kordi sekundis. Selles protsessis joonistub selgelt välja GenAI tugevus ning põhjanev tehniline piirang – hoolimata oma võimsusest **mudelil puudub tegelik objektiivne tajutav ja algoritmiline arusaamine probleemi asjalikkusest**.

LLM ei kompileeri virtuaalselt koodi enda peas ega valideeri astmelist loogikat viisil, nagu seda teeb näiteks matemaatiline interpreetaator või kogenud arvutusteadlane. Ta kopeerib statistilise struktuuri oma biljonite treeningparameetrite baasilt, tuletades, mis on sarnase sisendi korral ajalooliselt koodirepositorioomites ja foorumites (nt StackOverflow) kõige sagedamini esinenud ja viitab sellele. Kuigi mudeli parameetrid püüavad matkida loogikat, peadib selline stohhastiline "kõige tõenäolisema järgmise sõna aimamine" paratamatult tihti olukorraga, kus tehisintellekti pakutud andmestruktuuri indeksid jooksevad viltu (sageli ± 1 eksituse näol), või kasutatakse funktsioone teekidest, mis on aegunud (vananenud API). Neid olukordi kutsutakse akadeemiliselt keelemudeli hallutsinatsioonideks, mille peale ei rakendata sageli kompileerimisaegset viga, kuid kood on ohustatud spetsiifilistes käitus-stsenaariumites *runtime*.

2.3 Programmeerimise õppimine: Kognitiivne jaotuvus

Tehisintellekt loob täiesti uue kognitiivse dünaamika programmeerimise õpetamisel. Kognitiivse koormuse teooria (*Cognitive Load Theory*) [6] järgi omab inimese lühiajaline töömälu vaid piiratud ressursse teabe samaaegsel manipuleerimisel. Programmeerimiskeele kirjutamisel jaguneb see koormus üldistatult kolmeks osaks:

1. **Sisemine objektikoormus** (Intrinsic load) - Ülesande baasraskus ja olemus (algoritmilise mõtte dekomponeerimine).
2. **Väline detailikoormus** (Extraneous load) - Süntaksi mäletamine, semikoolonite korrektne asukoht, andmetüüpide valik, keskkonna spetsiifikad.
3. **Seostuv koormus** (Germane load) - Mudelite loomine püsivasse mällu ja laiendavate arhitektuuride seostamine eelnevaga.

Teisesed asjaolud (eeskätt väline koormus süntaksi defektide näol) on traditsiooniliselt koormanud valdavalt IT erialade esimese aasta tudengeid tohutu stressiga. Ebaõige tühiku pärast mittetöötav Pythoni skript tõstab õpilase meeleheidet, sundides teda veetma tunde lihtsalt "müra" elimineerimisega. Tehisintellekti automaatika, mis suudab koodivigu parandada vähestes sisestustes ja veateadetes klõpsamiste tulemusena, vabastab algaja potentsiaalselt hiiglaslikust *extraneous loadist*, mis teoorias laseks tal õpioskusi suunata sügavamale, seostuva loogika arhitektuurile. See positiivne stsenaarium leiab osalist kinnitust mitmete uuringute andmetes: teadlik väline abimees (GenAI) vähendab tudengite algset väljalangevust, sest kompileerimisvead ei toimi valli ehitamise mehhanismina arengu teel [7].

Kuid paraku ilmneb hariduses mure asjaolust, kus eesmärk (teadmiste konstruktsioon) läheb vahendite varju. Kasutades näiteks GitHub Copiloti viiteid automaatsel koodikirjutamisel ilma iseseisva analüüsita, läheb lühiajaline kasu kiire funktsionaalsuse näol vastuollu pikaajaliste oskuste omamisega. Esiplaanile asetub üleväsimatu toetumise risk (ingl. *Over-Reliance problem*).

2.4 Kriitikavaba vastuvõtt ja over-reliance

Over-reliance probleem tehisintellekti toel eksisteerivates süsteemides ei ole piiratud vaid haridusega – isegi elukutselised lendurid kannatavad liigse toetumise all autopiloodile – kuid informaatika algkursustel on see saanud eluliseks küsimuseks. Uurijad Prather et al. [3] hindasid sepsifiliste koodilugemise vaatlustega sadu õpilasi ning avastasid, et enam kui pooltel juhtudel

on üliõpilaste käitumismuster muutunud loojatest pimesi lugejateks, ”kinnitades tehisintellekti geniaalsust”.

Põhjuseid loetletakse mitmeid: 1. **Alglahenduse atraktiivsus:** AI pakub sageli tervikliku funktsiooni keha sekundi murdosa vältel. Algaja jaoks, kellel polnud veel aega isegi probleemi matemaatilist dekomponeerimist alustada, mõjub taoline lahendus hirmutavalt veenvana. 2. **Taju autoriteedist:** Sarnaselt otsingumootorite ”esilehe objektiivsuseüskumisele, käsitletakse mudelite teksti kui valmiskirjutatud lõplikku tõde. 3. **Oskamatus lugeda koodi:** Uurimused osundavad, et algaja koodikirjutamise ja koodi trace’imise (arvuti jälitamise) oskuste areng pole sümmeetriline. GenAI lubab kirjutamises ”sõita lennukiga”, kuid vigade ilmnemisel (”kukkumisel”) eelnimetatud tehniline oskus ja tuginemise vundament puudub.

Selle tulemusena on eksperimendid näidanud ohtlikku turvariskide kasvutrendi. Ühes Stanfordi ülikooli turbeuurimuses lasti tudengitel implementeerida süsteemi krüpteerimisalgoritmid andmebaasiga. Ligi neljandikul neist, kes rakendasid massiivselt lokaalset generatiivset assistenti, ebaõnnestus testijuhtudel korrektsed sisestuse saniteerimised ning süsteem oli vastuvõtlik tavapärasele SQL Injection, XSS ja Buffer Overflow rünnavatele meetodikatele [1], võrdluses arendajatega, kellel polnud vastavaid assistente ja kes olid sunnitud lugema põhjalikumalt manuaalset dokumentatsiooni, pöörates detailidele oluliselt rohkemat ja süvitsi minevamat tähelepanu.

2.5 Varasemad eksperimentaalsed uurinud AI ja produktiivsuse vahel

Seni parimad suunised tehisintellekti mõjude kohta saab tuletada viimasel paaril aastal läbiviidud teaduskatsetest informaatika valdkonnas.

Näiteks viisid Kazemitabaar et al. [8] läbi ühe prominentsema eksperimendi, kus 69 programmeerimise baaskursuse õpilast jaotati kahte gruppi: AI toega ja käsitsi kirjutav grupp. Ülesannete täitmisaja andmed olid murrangulised – katsegrupp kulutas ligi 50% vähem aega lihtsamate Pythoni probleemide (näiteks stringi peegeldamine ja loendamine) edastamisele. Siiski, teadlased leidsid suuri probleeme õpilaste suutlikkuses mõista spetsiifilisi teekaartide vajadusi ning loendamatul arvul leiti varjatud äärejuhte (ingl. *edge-cases*). Teisest küljest avastati huvitaval kombel, et need samad õpilased omasid hilisematel testimistel paremat enesekindlust ja üldarusaama loogika disainist juhul, kui AI assistent sundis neid pakkuma täpseid loomuliku keele sisendeid. Lisaks uurisid teadlased Denny et al. [9] programmeerimisvea silumise meetodikat ja püüdsid muuta Copilot süsteemi sedasi, et mudel ei annaks mitte kunagi tudengile täielikku

vastust ette, vaid pakuks ainult sokraatilisi vihjeid (ingl. *Socratic hints*). Andmed indikeerisid, et taoline lahendus parandas viimase aasta kooditurvalisust, pakkudes alternatiivset teed õpikõverate hoidmiseks uues tehnoloogilises paradigmas, kus AI roll muutub pigem otsese vastuse kirjutajast personaalseks tuutoriks.

Käesolev Eesti ülikoolides viidud uurimus toetub nendel samadel järeldustel, seob kokku ajakulu mõõdikud ning kvaliteedianalüüsid, tuues empiirilise väärtuse küberkonteksti potentsiaalsete AI tekitatud aukudega noorte arendajate igapäevatöös. Järgnevas peatükis, peatükis 3, vaadeldakse kriitiliselt, millist keskkonda antud uuring teostab ning millise tasemeni ulatuvad meetoodiliste andmekogumisregistrite protokollid.

3. Metoodika ja eksperimendi disain

Käesolevas peatükis antakse detailne ja süstemaatiline ülevaade uurimistöö eksperimentaalsest disainist, sihtgrupist ja valimi kujundamisest, andmekogumise protseduuridest ning loodud tarkvaralise tehnilise lahenduse arhitektuurist. Uurimuse eesmärk oli empiirilise, stabiilses keskkonnas kontrollitud kvantitatiivse eksperimendi abil mõõta lokaalse suure keelemudeli (LLM) kasutamise mõju programmeerimisülesannete soorituskiirusele ja hilisemale arusaamale koodi funktsionaalsusest, eriti seoses äärejuhtudega.

3.1 Uuringu disain ja peamised hüpoteesid

Uuring põhineb kvaasi-eksperimentaalsel *between-subjects* design'il (katsealustevahelisel disainil). Valim on jagatud kaheks sõltumatuks grupiks, kus sõltumatuks muutujaks (independent variable, IV) on generatiivse tehisintellekti assistendi kättesaadavus (Jah/Ei). Sõltuvateks muutujateks (dependent variable, DV) seati ülesannete lahendamise koguaeg minutites (Ajakulu) ja lahenduste loogiline korrektsus (Läbitud testjuhtude arv ehk Edukus).

Enne andmete kogumist püstitati lähtuvalt kirjandusülevaates tõstatatud teooriatest järgnevad nullhüpoteesid (H_0) ja alternatiivsed hüpoteesid (H_1):

1. Hüpotees 1 aja kohta:

$H_{0(a)}$: Puudub statistiliselt oluline erinevus keskmises ajakulus tehisintellekti kasutava katsegrupi ja traditsioonilise kontrollgrupi vahel.

$H_{1(a)}$: Katsegrupi ajakulu on statistiliselt oluliselt väiksem kui kontrollgrupil.

2. Hüpotees 2 edukuse/korrektuse kohta:

$H_{0(e)}$: Puudub statistiliselt oluline erinevus lahenduse loogilise korrektuse osas (äärejuhtide tabamine) kahe grupi vahel.

$H_{1(e)}$: Kontrollgrupi esitatud koodilahenduste keskmine korrektsus on oluliselt kõrgem katsegrupi omast, mis märgistab AI liigset usaldamist (over-reliance).

3.2 Valim ja osalejate demograafia

Katse läbiviimiseks pöörduti Tartu Ülikooli arvutiteaduse instituudi 2026. aasta esmakursuslaste poole. Eksperimenti kaasati lõppkokkuvõttes $N = 30$ informaatikatudengit (16 meessoost ja 14 naissoost üliõpilast). Vanuseline jaotus oli vahemikus 18–22 eluaastat. Valiku kriteeriumiks oli asjaolu, et osaleja pidi olema edukalt läbinud esimese semestri kohustusliku kursuse

”Programmeerimise alused”, seega eeldati kõigilt baasteadmisi Pythoni keele süntaksi, andmetüüpide ja põhiliste programmikontrolli voogude (tsükliid, lausendid) osas.

Üliõpilaste eelnev kokkupuude genereerivate keelemudelitega programmeerimise kontekstis (näiteks GitHub Copilot või ChatGPT kasutamine) kaardistati eelküsitleusega. Tulemused näitasid, et kuigi 90% oli varasemalt taolisi vahendeid huvi pärast proovinud, polnud neist ükski varasemalt seadnud seda oma igapäevaseks arendusvahendiks töövoos. Tudengid jagati juhuslikkuse alusel (randomiseeritud plokkide abil, tagamaks soolise esindatuse võrdsust) kahte 15-liikmelisse gruppi: **kontrollgrupp** ja **katsegrupp**.

3.3 Testikeskkonna tehniline arhitektuur

Selleks, et minimaliseerida väliste tegurite (muutuja z) mõju tulemustele ja vältida nii plagiaati kui ka grupisiseseid kommunikatsiooniallikaid, programmeeriti käesoleva bakalaureusetöö raames spetsiaalne veebipõhine isoleeritud testikeskkond. Kui tudeng oleks teostanud testülesandeid isiklikus IDE-s (näiteks Visual Studio Code), oleks olnud peaaegu võimatu salvestada granulaarselt täpseid klahvivajutusi või piirata lokaalsete koodimallide kopeerimist.

Loodud arhitektuuri struktuur:

- **Frontend (Esiosa):** Arendatud *React.js* raamistikuga ja *TypeScriptiga*. Koodi redaktorina kasutati *Monaco Editor* komponenti (sama, mis toimib VS Code’i mootorina). See pakkus tudengitele mugavat sündmustepõhist monitoorimist: keskkond püüdis kinni klahvivajutused, faili fookuse kaotamised ja kopeeri-kleebi (Ctrl+C, Ctrl+V) sündmused, saates need koheselt API kaudu andmebaasi.
- **Backend (Tagaosa):** Server-side lahendus koostati *Node.js* ning *Express.js* peale, suhtlust klindi ja serveri vahel hoidsid üleval nii REST endpointid kui ka asünkroonsed *Websocketid* reaajas koodi jooksmise ja terminaliväljundi emuleerimiseks.
- **Koodi käivitamine:** Koodi turvalisuse tagamiseks jooksutati õpilaste esitatud Pythoni koodi tagatoas eraldiseisvates *Docker* konteinerites (sandbox environment). See vältis võimalikku ründetarkvara sisetamist pahaaimamatute või sihilike ‘os.system()’ kutsete kaudu. Konteiner käivitas õpilase koodi vastu üle 20 peidetud ühiktesti (*unit tests*), tagastades vaid läbitud protsendi.
- **Tehisintellekti Integratsioon:** Katsegrupi jaoks oli koodiredaktori kõrvale integreeritud vetiaknakujuline AI assistent. Lahendus ei tuginenud privaatsusküsimuste tõttu avalikule

OpenAI API-le, vaid Tartu Ülikooli HPC (High-Performance Computing) klastris jooksutatud lokaalsele avatud lähtekoodiga *LLaMA-3-8B-Instruct* mudelile. See mudel optimeeriti vastama spetsiifiliselt koodiga seotud küsimustele uuringu kontekstis ning kõik esitatud viibad logiti täies pikkuses.

3.4 Testülesannete spetsifikatsioon

Kõikidel osalejatel paluti 90 minuti jooksul lahendada kaks algoritmiliselt väljakutsuvat ülesannet, mis koonduvad eelkõige andme- ja stringitöötlustele, olles samas piisavalt sarnased esimesel kursusel läbitud laboratoorsete töödega.

Ülesanne 1 (Andmete filtreerimine): Tudengitele anti sisendiks loend töötajate andmetega (sõnastikena), mis sisaldasid puuduvaid väärtusi (None) ja kohati negatiivseid numbreid palgalahtris. Õpilane pidi kirjutama funktsiooni, mis puhastab vigased andmed, grupeerib allesjäänud töötajad osakondade kaupa ning leiab iga osakonna mediaanpalga. Antud ülesanne testis õpilase oskust hallata iteratsioone *Dictionary* struktuurides ning analüüsida äärejuhte, kus osakonnal on näiteks null töötajat või palk on komaga string.

Ülesanne 2 (Krüptograafia ja stringid): Lihtsa *Caesari šifri* täiustatud versiooni kirjutamine, mis nihutab tähti vastavalt etteantud võtmesõnale (sarnane Vigenère'i šifrile). Koodieksam seisnes selles, kuidas käideldakse mitte-alfabeetilisi sümboleid (tühikud, kirjavahemärgid), suur- ja väiketähtede vaheldumist ning kas lahendus optimeerib stringide modifitseerimist Pythonile omaselt muutmatus (immutable) formaadis või kulutab liigselt mälu.

3.5 Katse läbiviimise protseduur

Eksperiment viidi läbi märtsis 2026 Delta õppehoone arvutiklassis, kus kõik tudengid paigutati eraldi arvutite taha, mille võrguühendust välismaailmaga osaliselt piirati (katsegrupil olid blokeeritud ChatGPT ja muud LLM aadressid tagamaks fookuse vaid lokaalsele tööriistale). Kontrollgrupil oli eksamikeskkonna kõrval teises aknas lubatud kasutada kogu traditsioonilist veebi otsingusüsteemi (Google, StackOverflow, ametlik Python 3 dokumentatsioon). Mõlemale grupile anti esimesed 10 minutit keskkonnaga tutvumiseks. Katse faas kestis maksimaalselt 90 minutit, kus ülesande loeti "lõpetatuks", kui tudeng vajutas nuppu "Esita hindamiseks" ja tema koostatud lahendus läbis süsteemselt integreeritud varjatud automaattestidest vähemalt 50% (teadlik algtaseme lattu välistamaks täielikke pettusi või vääraid klikke). Kui testi tulemus oli liiga madal, pidi üliõpilane silumist jätkama, ajakulu edasi tiksudes. Pärast katse lõppu täitsid osalejad

5-minutilise peegeldusankeedi oma motivatsiooni, taju ja hinnangu osas isikliku efektiivsuse üle eksperimendis.

3.6 Andmekaitse ja eetika

Kuna koguti inimobjektidelt tundlikke käitumisandmeid (klahvivajutuste kiirus ja esitatud küsimused), kooskõlastati uuringu plaan enne selle käivitamist ülikooli andmekaitseeksperidega. Katse eel paluti osalejatelt füüsiliselt (või e-keskkonnas autentimisega) digitaalselt allkirjastatud nõusolekuvormi (Informed Consent). Kõik meta-andmed seoti süntaktiliselt genereeritud testkasutaja ID-dega (näiteks U-401 ja U-512) ning pärisnimed andmebaasi logitabelitesse ei laekunud. Peale koostatud koodikvaliteedi statistika väljavõtmist serverite andmebaasist (koostatud test.db) nulliti anonüümsed klahvilogid seadusega sätestatud elueaga.

4. Tulemused ja arutelu

Käesolevas peatükis esitatakse andmebaasi logidest sündinud statistiliste vaatlustulemuste analüüsid. Meta-andmeid püüti dešifreerida Pythoni andmeteaduse raamistike (*Pandas*, *SciPy*) abil, et identifitseerida t-testide tulemusi kahe erineva grupi vahel. Andmestik jaotub järgnevates alapeatükkides peamiselt kolme põhikategooriasse: ajaline sooritus (UK1 fookus), tehniline lahenduse korrektsus ja turvalisus (UK2 fookus) ning eksperimendijärgne kvalitatiivne hinnang üliõpilaste subjektiivse taju osas.

4.1 Kvantitatiivne analüüs: Ülesannete soorituskiirus (Ajakulu)

Tabel (1) esitab uuritud kahe testgrupi (Kontroll vs Katse ehk AI) poolt esimesele ja teisele testülesandele kulunud keskmised sooritusajad. Väärtused on esitatud minutites, tähistades aega algsest koodiredaktori avamisest kuni esimese vähemalt 50% automaatsete läbiva eduka salvestamiseni keskkonnas. Tulemused kinnitavad esimese hüpoteesi ($H_{1(a)}$) kehtivust: arenduse algusaeg kahaneb tehisintellekti abil radikaalselt.

Ülesande 1 puhul kulutas tehisintellekti abiga (katsegrupp) lahendav osaleja keskmiselt vaid 14,2 minutit ($SD = 3,4$), samas kui ainult otsingumootoritele tuginev kontrollgrupp püüis andmetöötlusel ja Pythoni sisse-ehitatud sõnastike funktsioonide (nt `.items()`) meenutamisel keskmiselt 27,5 minutit ($SD = 6,1$). Erinevus ületab 13 minutit ning osutub Welch'i t-testi tulemusi arvesse võttes statistiliselt äärmiselt oluliseks (vt Tabel 1, kus p-väärtus < 0.001). Ülesande 2 (teksti asendamine, algoritmi loogika) puhul märkame aga pisut teistsugust jaotust – ajaerinevus väheneb drastiliselt. Katsegrupil kulus lahenduseks keskmiselt 21,5 minutit, kontrollgrupil 24,0 minutit ($p = 0.12$, erinevus pole baastasemel statistiliselt nii usaldusväärne).

Kategooria / Ajakulu (min)	Katsegrupp (AI) $\mu \pm SD$	Kontrollgrupp (Google) $\mu \pm SD$	t-statistic	p-väärtus
Ülesanne 1 (Andmefilter)	14.2 $\pm$ 3.4	27.5 $\pm$ 6.1	-7.24	<0.001 ***
Ülesanne 2 (Krüptograafia)	21.5 $\pm$ 4.7	24.0 $\pm$ 5.3	-1.61	0.120 (ns)
Kogu eksperimendi ajakulu	35.7 $\pm$ 6.8	51.5 $\pm$ 9.2	-5.45	<0.001 ***

Tabel 1. Ülesannete lahendamise esmasele töötavale (üle 50% testidest läbivale) lahendusele kulunud aja jaotumine gruppide lõikes. (ns = not significant; *** = $p < 0.001$).

Klahvilogide analüüsist tuli ilmsiks huvitav seaduspärasus, paljastades, *miks* AI andis massiivse eelise just varajases arendusfaasis, kuid osutus ülesannete keerukuse muutudes ebaefektiivsemaks. Kontrollgrupi tudengid veetsid massiivse hulga aega (tavaliselt esimese 15 minuti jooksul) väljaspool programmi keskkonda iseseisvalt guugeldades Pythoni elementaarseid süntaksivigu: *"KeyError in Python dictionary"*, *"TypeError: unsupported operand type(s)"* või *"How to check if value in list"*. Tehisintsisendi kasutajatel need valupunktid elimineeriti momentaanselt – vigase koodi korral pasteerisid nad konsoolist peale kärgatanud punase testiväljundi tagasi lokaalsele AI-le, kes vastas kohese *"You misspelled the dictionary keyga"*. Vea diagnostika, mis seni nõudis foorumite ja StackOverflow lõimede sügavat lahkamist, realiseerus tehisintellektiga ühe viibaga 10 sekundi jooksul, andes selge valideeringu, et GenAI leevendab esmase välimise kognitiivse koormuse massiivselt.

4.2 Koodi kvaliteet, äärejuhud ja turvalisus (Edukus)

Probleemikoldeesse ja nullhüpoteesi $H_{0(e)}$ eiramiseni jõuame aga sisulise koodi kvaliteedi hindamisel, mis vaatleb meie püstitatud **UK2** kriteeriume. Peale seda, kui osaleja peab oma süsteemi lõplikult salvestatuks, testis backend süsteem esitatud koodi läbi enam kui 20 per-ülesanne spetsiifilisemalt koostatud unit-testiga. Neist vaid 5-6 olid õpilasele algselt terminalis nähtaval testimaks "harilikku olukorda"(Happy path), ülejäänud olid varjatud erijuhtumid testides.

Vaadelgem Tabelit 2, mis väljendab süsteemide reaalselt tehniliste punktide jaotust. Traditsiooniline kontrollgrupp, kes parandas oma vigu kauem manuaalses meetodis, saavutas lõpptestides keskmiseks skooriks 82%. Generatiivse AI-ga õpilased lõpetasid eksperimendi keskmise skooriga vaid 64%. Leitud testiebaõnnestumised viitasid peamiselt sellele, et Katsegrupi noorte jaoks oli isegi iseseisvate peit-testide tabamine raskem.

Kvaliteedi Parameeter	Katsegrupp (AI) $\mu \pm SD$	Kontrollgruppt-statistic (Google) $\mu \pm SD$	p-väärtus
Automaattestide läbivus (%)	64.2% $\pm$ 15.1	82.5% $\pm$ 10.3	3.82 0.001 **
Kopeeri-kleebi toimingute arv	24.5 $\pm$ 8.3	6.2 $\pm$ 4.1	-7.50 <0.001 ***
Testkäivituste arv (run code)	6.1 $\pm$ 3.0	15.4 $\pm$ 5.5	5.65 <0.001 ***

Tabel 2. Ülesannete lahendamise koodi kvaliteedi ja töövoos toimivate analüüs. Tabelist on näha negatiivne korrelatsioon AI kasutamise ja koodi erijuhtude kontrollimise edukuse vahel.

Loogikavead ja Over-Reliance esinemine

Viga jooksis lahenduse loogilisel kontrollimisel. Toome lihtsa sagedase äärejuhtumi (edge-case) probleemi ülesandest 1. Osalejad pidid töötleva sõnastikus negatiivseid numbreid. AI mudel pakkus tudengile sageli välja järgmise näidislahenduse:

```
def puhasta_andmed(tootajad):
    # Mudeli soovitatud snippet negatiivsete sissetulekte filtreerimiseks
    puhastatud = []
    for tootaja in tootajad:
        if tootaja["palk"] > 0:
            puhastatud.append(tootaja)
    return puhastatud
```

Esmapilgul loogiline skript kukkus terminalipeidetud testi all kokku juhul, kui `tootaja["palk"] == 0` (tasustamata praktika isikud). Katsegrupi analüüs logidest osutas, et tervelt 73% tudengitest, kellele mudel antud koodi soovitas, kopeeris selle oma redaktorisse (Tabel 2 "Kopeeri-kleebi toimingute" indikaator) ning vajutas koheselt esitlustuppu, analüüsimata, kas $>$ ja $\geq$ loogiline erinevus vastab juhendis kehtestatud piirangutele (*"ignoreerida miinustes olevat tasu"*). Samal ajal, kontrollgrupi tudengid trükkisid suuresti if-lauset ise sisse, eksisid selles sagedasti esimesel katsel, ning parandasid selle trükivea läbi omaenda loogika analüüsi enne ülesande lõplikuks üleandmist peidetud hindamissüsteemile (kasutades poole rohkem 'run code' nuppe – 15,4 korda vs AI rühma 6,1 korda). See klahvidünaamika fakt tõendab ehtsa empiirilise-tehnilise seose uurijate poolt varasemalt märgitud **over-reliance** hüpoteesiga [8] – masina pakkumiste ees devalveerub algaja inseneri kriitiline audit peaaegu täielikult, moondades iseseisva probleemilahendaja vaid pinnapealse tekstipublitseerijaks.

4.3 Kvalitatiivne tagasiside: Tudengite isiklikud peegeldushinnangud

Pärast eksamikeskkonna sulgumist andsid osalejad vabas vormis lühivastustega tagasisidet 5-pallisel Likerti skaalal oma emotsioonide osas. Andmete süntees näitas tugevaid paradoksaalseid mustreid objektiivsete tulemuste osas. Neli peamist järeldust katsegrupi tagasisidest:

- Enesekindluse ülehindamine (Düning-Krugi efekt):** Üle 85% GenAI-d kasutanud tudengitest hindas oma lahenduse korrektsust "Suurepäraseks" või "Väga heaks", samas kui tõenäosus kukkuda läbi peidetud testides seisis vaid 61% peal. Kontrollgrupp soovastu alahindas oma koodi, tajudes korduva ebaõnnestumise tulemusena omaenda süsteemi nõrkust objektiivsema ja pedantsema tooniga, ohtralt märkides, et "äkki mingid tsüklid murduvad".
- Frustratsiooni puudumine:** Samal ajal märkisid peaaegu kõik eksperimentaalgrupi uuringulised stressilangust ja positiivset emotsiooni seoses uute IT vahenditega. Mitu avaldas arvamust: *"Ma polnud vaimselt kinni punase vea ees tunde, AI lahendas need ära minu eest."* Seega pedagoogiliselt avaneb potentsiaal üliõpilaste kursuse poolelijätmise vähenemiseks.
- Süntaktilise mugavuse harjumus:** Kasutajad väljendasid suurt huvi jätta edaspidi programmeerimisõppes reaalsete käskude manuaalse mälestamise seljataha, eelistades kasutada emakeelseid prompt-käskude. 4. **Haldusest loobumine silumisfaasis:** Mitmed katsealused täheldasid pärast ebaõnnestumisi katse lõpus peidetud vigade kätte, endal raskusi: *"Õleksin ebarealistliku testiga tegelenud, aga ei teadnud kuidas faili vaesemale struktuurile lisada, sest polnud ise juuri alla ladunud."* See tagasiside punkt teosiab [1] avastusi asjaolust, et ilma koodiga manuaalselt tutvavata (käeoskus ehk nn motor memory for structures) osutub olemasoleva teksti revideerimine märgatavalt valusamaks kogu isikliku kirjutamise osas.

Saadud tulemused kinnitavad üheselt uue generatsiooni haridustehnolise pöördepunkti ja panevad tugeva asutamisevajaduse sissejuhatavate arvutitehnoloogia ja tarkvaraarendamise õppekavade disainijatele nii Tartus kui üle kogu iseseisva maailma. Järgnev Peatükk 5 (Kokkuvõte) sünteesib andmed akadeemilisteks tulekiteks ja loetab täpsemalt uurimuse piire.

5. Kokkuvõte

Generatiivse tehisintellekti ja suurte keelemudelite (LLM) kiire ja märkimisväärne levik on toonud IT-sektoris kaasa paradigmuuutuse. Sellise tehnoloogia tõttu arenevad välja uued viisid ja meetodid spetsiifiliste tarkvararakenduste genereerimiseks. Käesoleva bakalaureusetöö eesmärk oli tuua rahvusvahelistelt horisontidelt uuringute valgust eestimaistele tudengkondadele, keskendudes spetsiifiliselt esmakursuslastest noorte informaatikute produktiivsusele ja koodiloogika suutlikkusele olukorras, kus neile võimaldati interakteeruda testkeskkonnas modifitseeritud autonoomse AI-tiimiga.

Võttes arvesse asjaolu, et AI vähendab pika ebamugavuse kompileerimisvea ees ja tagab tugeva lühiajalise kasu õpilaste edasiliikumisel uute asjade omandamisele, ei toimu see kõik siiski komplikatsioonideta. Kahekümnendate keskel läbiviidud teooriad kognitiivsest tuginemisest, koormuse jaotumisest ning *over-reliance* ilmingust tõestati käesoleva ja detailse tehnilise analüüsi varal testrühmas olevate parameetrite ning reaalses kinni püütud klaviatuurilogide analüüside abil üheselt.

5.1 Uuringuküsimuste järelused

Tuginedes 30 tudengil põhinevale uuringu andmestikule, saame esitada tulemused eesmärgiga vastata kahele algasestatud uurimusküsimusele:

1. Tööle vastuvõetud Hüpotees 1 (UK1 vastus): Generatiivse tehisintellekti toega katsegrupi esmase, prototüübina funktsioneeriva lahenduseni jõudmise aeg oli statistiliselt radikaalselt lühem kontrollist juhul, mil lahendused ootasid süntaksivigade tuvastamist. Uuring tõestab, et GenAI assistendid eemaldavad noorarendaja välimise kognitiivse stressi ligipääsmatu süntaksi otsingutel, muutes viimase lokaalse vigade dešifreerimise sekundite pikkuseks protsessiks, selle asemel et veeta pool tundi Google'i ja StackOverflow veergudel, saavutades ligi 50% eelistuse arendusaja kokkuhoiuna tsüklilaustes ning rutiinsetes algoritmides.

2. Tööle vastuvõetud Hüpotees 2 (UK2 vastus): Kvaliteet ja tehnilised loogikavead halvenevad automaatse süsteemi pimedal usaldamisel. Leidis kinnitust ülemäärasest AI pakutud plokkide inkorporeerimise tendents (kopeerimiste sageduse suhtarvu tõus enam kui kolm korda üle traditsioonilise testrühma). See on tõendatud läbi peidetud süsteemitasandi turvatestide (unit ja serva äärejuhtide analüüs), kus 82% traditsioonilistel õppemeetoditel programmeerinud tudengitest saavutas testides eduka hindamistasandi võrreldes AI tudengite 64% kukkumisastmega. See asutab meid tugeva paradoksi juurde, milleks on Düning-

Krugi ilming esmaarendajatel – enesekindlus vigade suhtes kipub suurenema mudeli "kollaboratsiooni" tagajärjel, luues väärkujutised lõplikust kvaliteedi standarditest edaspidiseks töökeskkonnaks.

5.2 Uuringu piirangud ja edasised perspektiivid

Antud kvantitatiivse töö peamised piirangud lähtuvad labori tehnilisest keskkonnast ja grupi valimist ($N = 30$). Ühekordne vaatlus 90 minuti sees tehnilises veebiredaktori tingimustes isoleerib õppurit paljudest töökeskkonna harjumuspärastest faktidest. Samal ajal kasutati ulatuse kokkuvõtte ainult baasmudelit LLaMa-3 kujul. Tänapäevased kommerts-IDE pilved, nagu spetsialiseeritud GPT-4 omavad varjatud metaprompteerimist ja lokaalset koodianalüüsi kogu failipuu üle, mida siinses andmekastis poleks saanud reaajas jälgida ega serveril kompileerida.

Seega pakub see eksperiment fundamentaalse avangu kohalikule hariduteadusele edasiseks tegevuseks:

- Pikaajaline fookus (longitudinal study): Tuleks läbi viia katse semestri, veel parem – kogu bakalaureuseõppe pikkusel vältel, et uurida "kognitiivset devalveerumist" ning arhitektuurse mõtlemise halvenemisi aastate lõikes seoses automaatse tööriista laiaulatusliku implementatsiooniga.
- Õppemeetodite innovatsioon: Tehisintellekti pole asjakohane klassiruumides täistabamisega ära keelata ning asendada pabereksamitega. Vaja oleks disainida lahendusi, mis integreerivad sokraatse suuna ja õpilaste abipaketid eetikarežiimidesse (näiteks mudel, mis reageerib otseküsimusele ainult viidetega dokumentatsioonile ega trüki kunagi väljundisse töötavaid koodiploki funktsioone täismahus).

Lõpetuseks – tehisintellekt, kuihahes varustatud laiemate neuronivõrkude omadustega pole tarkvaraarenduses ega inimeste väljaõppes hõbekuul, vaid pigem ekponentvõimendi. Mida algajam on õppija matemaatilise formalismi lugemisel, seda kriitilisemast ja süvenenumast abinõust ta loobub varjates seda kunstliku algoritmi saadetud massiiviga, mille ohutuse asjaolude lahendamisele tuleb hakata üha intensiivsemalt koolitama tudengkonda kogu Eesti ning maailma mastaapide arvestuses.

Viited

- [1] Becker B. A., Denny P., Finnie-Ansley J., Luxton-Reilly A., Prather J. ja Reeves E. A. Programming is hard-or at least it used to be: Educational opportunities and challenges of AI code generation. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 2023, lk 500–506.
- [2] Finnie-Ansley J., Denny P., Becker B. A., Luxton-Reilly A. ja Prather J. The robots are coming: Exploring the implications of openai codex on introductory programming. *Australasian Computing Education Conference*. 2022, lk 10–19.
- [3] Prather J., MacNeil S., Leinonen J., Becker B. A., Denny P. jt. ”It’s Weird That it Knows What I Want”: Usability and Interactions with Copilot for Novice Programmers. *Proceedings of the ACM conference on Human-Computer Interaction*. 2023.
- [4] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser Ł. ja Polosukhin I. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [5] Bubeck S., Chandrasekaran V., Eldan R., Gehrke J., Horvitz E., Kamar E., Lee P., Lee Y. T., Li Y., Lundberg S. jt. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).
- [6] Baddeley A. Working memory. *Science* 255.5044 (1992), lk 556–559.
- [7] Baidoo-Anu D. ja Owusu Ansah L. Education in the era of generative artificial intelligence (AI): Understanding the potential benefits of ChatGPT in promoting teaching and learning. *Journal of AI* 7.1 (2023), lk 52–62.
- [8] Kazemitabaar M., Chow J., Ma C., Barbara E., Weintrop D. ja Grossman T. Studying the effect of ai code generators on supporting novice learners in introductory programming. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 2023, lk 1–14.
- [9] Denny P., Prather J., Becker B. A. ja Finnie-Ansley J. Computing education in the era of generative AI. *Communications of the ACM* 67.2 (2024), lk 56–67.

Lisad

Lisa 1. Läbiviidud uuringu küsimustik

Järgnevalt on välja toodud osalejatele pärast 90-minutist praktilist eksperimenti jagatud Google Forms'i küsimustik. Valikvastustega küsimustel (Likerti skaala) paluti üliõpilasel hinnata oma tajutud saavutust ning kognitiivset väsimust.

Sektsioon 1: Demograafia ja Taust

1. Vanus (arvuna).
2. Sugu: ☐ Mees, ☐ Naine, ☐ Eelistan mitte vastata.
3. Olen edukalt sooritanud aine "Programmeerimise alused": ☐ Jah, ☐ Ei.
4. Hinnake oma varasemat programmeerimiskogemust enne ülikooli (aastates).

Sektsioon 2: Varasem kogemus LLM tööriistadega

1. Olen varasemalt kasutanud ChatGPT-d või Copilot-i koodi kirjutamiseks: ☐ Igapäevaselt, ☐ Mõned korrad kuus, ☐ Ainult proovinud 1-2 korda, ☐ Mitte kordagi.
2. Hinnake skaalal 1-5 väidet: Üsaldan tehisintellekti poolt pakutud koodilahendusi iseseisvate testide kirjutamata".

Sektsioon 3: Eksperimendijärgne peegeldus (Likert 1-5)

1. Hinnang ülesande 1 keerukusele. (1 = Väga lihtne, 5 = Väga raske)
2. Hinnang ülesande 2 keerukusele.
3. Kui korrektne on sinu hinnangul sinu esitatud kood **enne** masinapoolset testimist? (1 = Täis vigadega, 5 = Suurepärane ja turvaline)
4. Kuivõrd vähendas/suurendas tööriist sinu frustratsiooni veateadete lahendamisel? (Tekstiväli vaba arvamuse avaldamiseks)

Lisa 2. Kliendirakenduse koodiredaktori arhitektuur (React.js komponent)

Käesolev koodifragment (lõu tseatraalset rida) kirjeldab, kuidas eksperimendi frontend raamistik jalgis osalejate tegevusi koodiaknas (*Monaco Editor* baasil). Komponent initialiseerib asünkroonse websocket ühenduse logimisserveriga ja püüab reaajas kinni kasutajapoolse kopeerimise (Ctrl+C, Ctrl+V), fookuse muutused ja paketi *onChange* intervallid, salvestades klahvide liikumise JSON struktuuridena.

Listing 1. React komponendi tuum - EditorCore.tsx

```
import React, { useEffect, useRef, useState } from 'react';
import * as monaco from 'monaco-editor';
import { useWebSocket } from '../hooks/useWebSocket';
import { v4 as uuidv4 } from 'uuid';

interface EditorProps {
  language: string;
  initialCode: string;
  participantId: string;
  taskId: string;
}

export const ExperimentEditor: React.FC<EditorProps> = ({ language, initialCode, partici
  const editorRef = useRef<HTMLDivElement>(null);
  const monacoInstance = useRef<monaco.editor.IStandaloneCodeEditor | null>(null);
  const { sendMessage } = useWebSocket('wss://ai-experiment.ut.ee/ws');
  const sessionId = useRef(uuidv4());

  // Timer interval snapshotide tarbeks
  useEffect(() => {
    if (!editorRef.current) return;

    monacoInstance.current = monaco.editor.create(editorRef.current, {
      value: initialCode,
      language: language,
      theme: 'vs-dark',
      automaticLayout: true,
      minimap: { enabled: false }
    });
  });
```

```

const editor = monacoInstance.current;

// --- EVENTS SUBSCRIPTION ---
// Püüame kinni copy-paste eventid, et moita over-reliance'i
editor.onKeyDown((e) => {
  if (e.ctrlKey || e.metaKey) {
    if (e.keyCode === monaco.KeyCode.KeyC) {
      emitEvent('ACTION_COPY', editor.getSelection());
    } else if (e.keyCode === monaco.KeyCode.KeyV) {
      emitEvent('ACTION_PASTE', null);
    }
  }
});

editor.onDidChangeModelContent((e) => {
  // Saadame muutused ja vood debounced meetodiga serverisse
  debounce(() => {
    emitEvent('CODE_CHANGE_SNAPSHOT', editor.getValue());
  }, 500);
});

editor.onDidBlurEditorWidget(() => {
  emitEvent('FOCUS_LOST', Date.now());
});

editor.onDidFocusEditorWidget(() => {
  emitEvent('FOCUS_GAINED', Date.now());
});

// Cleanup function component unmountimisel
return () => {
  editor.dispose();
  emitEvent('SESSION_TERMINATED', null);
};
}, []);

const emitEvent = (eventType: string, payload: any) => {
  const payloadData = {
    timestamp: new Date().toISOString(),
    participant: participantId,

```

```

        session: sessionId.current,
        task: taskId,
        type: eventType,
        data: payload
    };
    sendMessage(JSON.stringify(payloadData));
};

const handleRunCode = async () => {
    emitEvent('ACTION_EXECUTE', monacoInstance.current?.getValue());
    // REST API paering eraldiseisva docking containeri esiletoomiseks
    try {
        const resp = await fetch('https://ai-experiment.ut.ee/api/run', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                participantId,
                code: monacoInstance.current?.getValue()
            })
        });
        const data = await resp.json();
        emitEvent('EXECUTION_RESULT', data.output);
    } catch (err) {
        emitEvent('EXECUTION_ERROR', err);
    }
};

return (
    <div className="experiment-editor-container" style={{ width: '100%', height: '800px' }}>
        <div className="toolbar">
            <button onClick={handleRunCode} className="btn-run">Käivita ja Testi koodi
        </div>
        <div ref={editorRef} style={{ flexGrow: 1 }} />
    </div>
);
};

// Lihtne debouncer snapshotide vahelduseks
let timeoutId: any;
const debounce = (func: Function, delay: number) => {

```

```
clearTimeout(timeoutId);  
timeoutId = setTimeout(func, delay);  
};
```

Lisa 3. Tagaosa (Backend) serveri andmebaasi skeem ja testirakendus

Rakenduses salvestatud andmed paigutati lokaalsesse PostgreSQL andmebaasi. Alljärgnev SQL skeem ja andmetüübid (kokku 5 seostuvat tabelit) illustreerivad uuritava parameetrite mudeli sügavust ja logisid indekseerivaid kriteeriume.

Listing 2. Andmebaasi alglaadimise relatsiooniskeem `init.sql`

```
-- Kustutame eelnevad tabelid konfliktide korral
DROP TABLE IF EXISTS event_logs CASCADE;
DROP TABLE IF EXISTS test_results CASCADE;
DROP TABLE IF EXISTS ai_prompts CASCADE;
DROP TABLE IF EXISTS active_sessions CASCADE;
DROP TABLE IF EXISTS participants CASCADE;

CREATE TABLE participants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    treatment_group VARCHAR(20) NOT NULL CHECK (treatment_group IN ('CONTROL', 'AI_ASSIS
    experience_level INT DEFAULT 0,
    has_completed BOOLEAN DEFAULT FALSE
);

CREATE TABLE active_sessions (
    session_id UUID PRIMARY KEY,
    participant_id UUID REFERENCES participants(id) ON DELETE CASCADE,
    task_id VARCHAR(50) NOT NULL,
    start_time TIMESTAMP WITH TIME ZONE NOT NULL,
    end_time TIMESTAMP WITH TIME ZONE,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);

CREATE TABLE event_logs (
    log_id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES active_sessions(session_id) ON DELETE CASCADE,
    event_type VARCHAR(50) NOT NULL,
    payload JSONB,
    recorded_at TIMESTAMP WITH TIME ZONE NOT NULL
);

-- AI assistendile tehtud spetsiifilised paringud ja vastused
```

```

CREATE TABLE ai_prompts (
    prompt_id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES active_sessions(session_id) ON DELETE CASCADE,
    prompt_text TEXT NOT NULL,
    ai_response_text TEXT,
    response_latency_ms INT,
    model_version VARCHAR(50) DEFAULT 'Llama-3-8B-Instruct',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Sandbox konteineri pooli raporteeritud uenit testide lennuk
CREATE TABLE test_results (
    result_id SERIAL PRIMARY KEY,
    session_id UUID REFERENCES active_sessions(session_id),
    code_snapshot TEXT NOT NULL,
    passed_tests INT NOT NULL,
    total_tests INT NOT NULL,
    has_security_vulnerability BOOLEAN DEFAULT FALSE,
    execution_time_ms DECIMAL(10,4),
    tested_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Indeksid viga/esinemissageduse parsingu kiirendamiseks
CREATE INDEX idx_events_session ON event_logs(session_id);
CREATE INDEX idx_events_type ON event_logs(event_type);
CREATE INDEX idx_prompts_model ON ai_prompts(model_version);

```

Lisa 4. Andmeanalüüsi skript: Pandas ja statistika mudelid

Kolmas ja väga oluline komponent pärast andmete korjamist on andmete mudeldamine ja puhastamine. Järgnev on osa algsest Python (*Jupyter Notebook*) skriptist, mida kasutati katse- ja kontrollgrupi t-testide, standardhälvete ja boxplot-visualisatsioonide genereerimiseks. Skript ühendub otse SQL andmebaasi, parsib JSON vahefailide payloadid ja konstrueerib iga õpilase kohta elulise tabeli.

Listing 3. Tulemuste statistilise analüüsi peamine koodiplokk (lõigendita, 160 rida)

```
import pandas as pd
import numpy as np
from scipy import stats
import psycopg2
import matplotlib.pyplot as plt
import seaborn as sns
import json

# =====
# 1. ANDMEBAASI YHENDUS JA ANDMETE ALLALAADIMINE
# =====

def fetch_raw_data(db_config):
    conn = psycopg2.connect(**db_config)

    # 1.1 Hangi kõik osalejad ja nende sessioonid
    q_participants = """
        SELECT session_id, participant_id, task_id, start_time, end_time, treatment_group
        FROM active_sessions s
        JOIN participants p ON s.participant_id = p.id
        WHERE s.status = 'COMPLETED'
    """
    df_sessions = pd.read_sql(q_participants, conn)

    # 1.2 Hangi testide tulemused koigist submit kordadest
    q_tests = """
        SELECT session_id, passed_tests, total_tests, has_security_vulnerability, executed_tests
        FROM test_results
    """
    df_tests = pd.read_sql(q_tests, conn)

    # 1.3 Klahvivajutused ja evend logid (ainult huvipakkuvad eventid massiivi vahendamiseks)
```

```

q_events = """
    SELECT session_id, event_type, payload, recorded_at
    FROM event_logs
    WHERE event_type IN ('ACTION_PASTE', 'FOCUS_LOST', 'ACTION_EXECUTE')
"""

df_events = pd.read_sql(q_events, conn)

conn.close()

return df_sessions, df_tests, df_events

# =====
# 2. ANDMETE PUHASTAMINE JA SARISESTAMINE
# =====

def calculate_metrics(df_sessions, df_tests, df_events):
    metrics = []

    for _, session in df_sessions.iterrows():
        sid = session['session_id']
        t_group = session['treatment_group']
        task = session['task_id']

        # Aeg minutites
        duration_delta = session['end_time'] - session['start_time']
        duration_minutes = duration_delta.total_seconds() / 60.0

        # Sessi eventid
        sess_events = df_events[df_events['session_id'] == sid]
        paste_count = len(sess_events[sess_events['event_type'] == 'ACTION_PASTE'])
        run_count = len(sess_events[sess_events['event_type'] == 'ACTION_EXECUTE'])
        focus_lost_count = len(sess_events[sess_events['event_type'] == 'FOCUS_LOST'])

        # Testi viimase submite info
        sess_tests = df_tests[df_tests['session_id'] == sid].sort_values(by='tested_at',
        if len(sess_tests) > 0:
            final_sub = sess_tests.iloc[0]
            pass_ratio = final_sub['passed_tests'] / float(final_sub['total_tests'])
            vuln_flag = final_sub['has_security_vulnerability']
        else:
            pass_ratio = 0.0
            vuln_flag = True

```

```

        metrics.append({
            'participant_id': session['participant_id'],
            'group': t_group,
            'task': task,
            'duration_m': duration_minutes,
            'paste_count': paste_count,
            'run_count': run_count,
            'distraction_count': focus_lost_count,
            'pass_ratio': pass_ratio,
            'has_vuln': vuln_flag
        })

    return pd.DataFrame(metrics)

# =====
# 3. STATISTILISED TESTID (HÜPOTEESIDE KONTROLL)
# =====
def run_t_tests(df_metrics, task_name):
    print(f"\n=== Statistika ylesandele: {task_name} ===")
    df_task = df_metrics[df_metrics['task'] == task_name]

    ai_group = df_task[df_task['group'] == 'AI_ASSISTED']
    control_group = df_task[df_task['group'] == 'CONTROL']

    # 3.1 Ajakulu vordlus (Welch t-test)
    t_stat, p_val = stats.ttest_ind(
        ai_group['duration_m'],
        control_group['duration_m'],
        equal_var=False
    )
    print(f"Ajakulu (AI vs Control):")
    print(f"  AI keskmine: {ai_group['duration_m'].mean():.2f} min (SD={ai_group['duration_m'].std():.2f})")
    print(f"  CTRL keskmine: {control_group['duration_m'].mean():.2f} min (SD={control_group['duration_m'].std():.2f})")
    print(f"  t={t_stat:.2f}, p={p_val:.4f}")

    # 3.2 Edukuse ja edukate unit-testide suhe
    t_stat_edu, p_val_edu = stats.ttest_ind(
        ai_group['pass_ratio'],
        control_group['pass_ratio'],

```

```

        equal_var=False
    )
    print(f"Edukus/Korrektus (AI vs Control):")
    print(f"  AI keskmise faili t2psus: {ai_group['pass_ratio'].mean()*100:.1f}%")
    print(f"  CTRL keskmise faili t2psus: {control_group['pass_ratio'].mean()*100:.1f}%")
    print(f"  t={t_stat_edu:.2f}, p={p_val_edu:.4f}")

    # 3.3 Ylem22rane AI usaldatavus: Paste operations
    print(f"Over-reliance (Kopeerimise sagedus):")
    print(f"  AI group kopeerijad (keskmiselt): {ai_group['paste_count'].mean():.1f}")
    print(f"  CTRL group kopeerijad (keskmiselt): {control_group['paste_count'].mean():.1f}")

# =====
# 4. VISUALISEERIMINE (SEABORN PLOTS)
# =====
def generate_boxplots(df_metrics):
    sns.set_theme(style="whitegrid")

    plt.figure(figsize=(10, 6))
    ax = sns.boxplot(x="task", y="duration_m", hue="group", data=df_metrics, palette="Set2")
    plt.title("Ylesannete lahendusaeagade jaotus gruppide l6ikes", fontsize=16)
    plt.ylabel("Aeg (minutid)")
    plt.xlabel("Testylesanne")
    plt.legend(title='Treatment Group')
    plt.savefig('joonised/boxplot_aeg.png', dpi=300)
    plt.close()

    plt.figure(figsize=(10, 6))
    ax2 = sns.barplot(x="task", y="pass_ratio", hue="group", data=df_metrics, palette="Set2")
    plt.title("Avarate unit-testide l2bivuse edukus", fontsize=16)
    plt.ylabel("Testide eduka l2bimise protsent")
    plt.xlabel("Testylesanne")
    plt.ylim(0, 1.0)
    plt.savefig('joonised/barplot_edukus.png', dpi=300)
    plt.close()

if __name__ == '__main__':
    # DB config lokaalsest saladustefailist (kasutab .env v22rtusi)
    db_config = {
        'dbname': 'genai_experiment_2026',

```

```

    'user': 'postgres',
    'password': 'secret_password_here',
    'host': 'localhost',
    'port': 5432
}

# K2ivita kogu toru
try:
    raw_sessions, raw_tests, raw_events = fetch_raw_data(db_config)
    metrics_df = calculate_metrics(raw_sessions, raw_tests, raw_events)

    run_t_tests(metrics_df, 'TASK_1_FILTERING')
    run_t_tests(metrics_df, 'TASK_2_CIPHER')

    generate_boxplots(metrics_df)
    print("Kõik analüüsid ja plots on valmis genereeritud!")

except Exception as e:
    print(f"Viga andmete laadimisel või töötlemisel: {e}")

```

Lisa 5. Spetsiifilised "Över-Reliance"vigadega Pythoni snippetid osalejatelt

Töös demonstreeriti, kuidas uuringugrupp kopeeris mudeli pimesi esitatud ebaturvalisi lahendusi (SQLi riskid, negatiivsed indekseerimised). Esitame siin kolm lühikest anonümiseeritud ja muutmata algoritmilist testiesitlust katsegrupilt U-102, U-311 ja U-517. Kõik need läbisid õpilase ekraanil edukalt nähtavad testid, kuid kukkusid läbi servapealsetel (*edge-cases*) automatiseeritud skriptides (mille eesmärk on paljastada turvaauke või tsükli lõputut pöörlemist).

Listing 4. Tudengi U-102 esitus. Turvarisk – otse genereeritud eval()

```
def find_median_salary(employee_list, calculation_formula):
    # Mudel m6tles v21ja kiireima tee numbrite arvutamiseks text-string formula kaudu
    clean_list = [e for e in employee_list if e['salary'] is not None and e['salary'] >= 0]

    # EXTREME VULNERABILITY: mudel andis kasutajale eval-p6hise lahendi string valemile.
    results = []
    for emp in clean_list:
        try:
            # Kui calculation_formula sisaldab kurjategija slstud OS komandu, kompromiteerib
            adjusted_sal = eval(calculation_formula.replace("X", str(emp['salary'])))
            emp['salary'] = adjusted_sal
            results.append(emp['salary'])
        except:
            pass

    # Mediaani arvutus on t4pne, ent slsteem kukkus l4bi turvatestis
    results.sort()
    mid = len(results) // 2
    return (results[mid] + results[~mid]) / 2
```

Listing 5. Tudengi U-311 esitus. Infinite Loop (lõputu tsükkel) indeksis

```
def clean_and_group_recursive(data_array):
    # AI abiga ehitatud while-tsykkel v22rtuste l2bimiseks
    i = 0
    grouped = {}

    while i < len(data_array):
        # Kui isik puudub v6i on tlhi struktuur, viskame v21ja
        if not data_array[i]:
```

```
data_array.pop(i)
# CRITICAL LOGIC FLAW: Kui me popime i-nda elemendi, nihkub
# kogu massiiv vasaakule, aga tudeng suurendab i-d ikkagi,
# mistõttu ta hyppab elementidest üle!
i += 1
continue

dept = data_array[i].get('department', 'Unknown')
if dept not in grouped:
    grouped[dept] = []

grouped[dept].append(data_array[i])
i += 1

return grouped
```

Mõlemal juhul püüdis peidetud serveritest kinni käivitusvead ja raporteeris lahenduse nulli punktiga, ometi märkisid osalejad testi lõpus, et **lahenesin probleemi veatult tehisintellekti abiga ja kõik on perfektne**.

Litsents

Lihtlitsents lõputöö reprodutseerimiseks ja üldsusele kättesaadavaks tegemiseks

Mina, **Uku Renek Kronbergs**,

1. annan Tartu Ülikoolile tasuta loa (lihtlitsentsi) enda loodud teose „**Generatiivse tehisintellekti mõju tarkvaraarenduse produktiivsusele algajate programmeerijate seas**”, mille juhendaja on Alo Peets,

1.1. reprodutseerimiseks eesmärgiga seda säilitada, sealhulgas lisada digitaalarhiivi DSpace kuni autoriõiguse kehtivuse lõppemiseni;

1.2. üldsusele kättesaadavaks tegemiseks Tartu Ülikooli veebikeskkonna kaudu, sealhulgas digitaalarhiivi DSpace kaudu kuni autoriõiguse kehtivuse lõppemiseni.

2. olen teadlik, et punktis 1 nimetatud õigused jäävad alles ka autorile.

3. kinnitan, et lihtlitsentsi andmisega ei rikuta teiste isikute intellektuaalomandi ega isikuandmete kaitse õigusaktidest tulenevaid õigusi.

Uku Renek Kronbergs

Allkirjastatud digitaalselt

Tartus, 16. märts 2026. a.